

IRIS Studio: the cinema robot arm of the thesis as a browser simulator, with a servo zoom lens, a planned shot library and a learning loop

Jerry (Qilong) Cheng

September 2026

Abstract

IRIS is the six-axis cinema robot arm of the author’s MEng thesis [1]: Unitree GO-M8010-6 actuators, an offset-axis layout, 0.7 m of reach and a visuomotor policy that learned camera moves from demonstrations. This document records the studio built on top of it in September 2026: the thesis’s MuJoCo kinematic model driven in the browser, a desk with product sets, a Sony α 7R III with a 16–150 mm zoom whose zoom and focus rings are servo-driven, a thin-lens depth-of-field model of the feed, 23 preset shots (pans, dollies, orbits, arcs, dolly zooms, rack focus, object tracking) defined as parametric lens paths, a damped-least-squares solver with a tool frame at the lens’s entrance pupil, and a demonstration-recording and policy-training loop. Every preset is planned at 30 Hz: the maximum joint speed over the library is $122^\circ/\text{s}$ and the peak inverse-dynamics torque 14.4 N m against the motors’ 23.7 N m, with 0 unreachable samples. The dolly zooms hold the subject’s silhouette width to a coefficient of variation of 0.1% and 0.1% where a plain dolly changes it by 14.6%, and a pixel measurement off the rendered clips confirms a 2.4% band against 38.5%. The videos on the web page were recorded from the same simulator; the executed telemetry (5739 frames over 23 shots) shows a mean lens position error of 0.26 mm and a median aim offset of 0.05% of the frame. The presets were then re-solved against a framing law rather than placed by eye: with the distance fixed by what the arm can reach, the focal length follows from how much of the frame the subject should fill, and reach, joint speed, clearance from the products and the lens’s minimum focus distance are checked along every path. That removed the 9 of 23 presets that had violated one of those limits — five swept the lens through a product and one push-in ended 74 mm from a coffee cup — and a measurement of the pictures themselves cut the frame area clipped to white from 11.6% to 3.7%. Two further experiments run in the browser itself: object tracking from the rendered feed (Section 3.6) and a policy trained on the studio’s own demonstrations (Section 3.7). All figures are regenerated by `experiments/make_figures.py`.

Keywords: cinema robotics, inverse kinematics, camera control, zoom lens, dolly zoom, imitation learning, browser simulation.

1 Overview

The thesis built IRIS, characterised it and learned camera moves from demonstration (VisionJoint-Planner: a five-frame window of RGB and joint states and a goal point in, the next five joint states out, trained on 17,897 samples). Its simulation lived in MuJoCo on a workstation and its camera was a RealSense. The studio described here moves the arm to the web page `iris.html` and its full-screen app `iris_studio.html`, adds the camera the next IRIS will carry, and turns cinematography into a library of planned, validated presets that a visitor can play, edit and learn

from. The Python side (`experiments/irisstudio`) and the JavaScript side (`assets/js/iris`) implement the same kinematics and the same shot rules; the planner validates in Python what the studio executes in the browser, and the browser’s own execution is logged back for Section 3.5.

2 Method

2.1 Kinematics from the MuJoCo tree

`iris_model.json` is exported from `mujoco_sim/assets/iris.xml`: for every body its fixed pose in the parent (position, quaternion), its hinge joint (axis, range) and its mesh. MuJoCo composes $T_{\text{world},b} = T_{\text{world,parent}} T(\mathbf{p}_b, \mathbf{q}_b) R(\hat{\mathbf{a}}_b, q_j)$, and both the Python and the JavaScript forward kinematics do the same; `tests/test_kinematics.py` checks positions and rotations of four bodies against `mj_forward` to 10^{-9} over 50 random configurations, and the geometric Jacobian against `mj_jacBody`. Joint 1 at zero puts the arm’s plane along $+y$; -90° faces the products. Joints 4 ($\pm 180^\circ$) and 6 ($\pm 360^\circ$) are continuous-rotation motors: the solver leaves them unclamped and every path difference is taken modulo 2π (an early planner counted a $\pm 180^\circ$ wrap as a 358° jump).

The tool frame. The mount’s $+z$ is the optical axis and its $-y$ is up (the convention of the thesis’s look-at controllers). The camera body and lens are modelled to scale; the lens axis passes through the mount origin and the entrance pupil sits $\mathbf{o} = (0, 0, 0.14\text{m})$ ahead of it, so the tool frame is the mount frame translated by $R\mathbf{o}$. A first version aimed the mount origin and put the lens 2cm beside it, which placed the subject 10% off centre at close range; aiming the tool frame removed it. All shot positions are lens positions.

2.2 Inverse kinematics

Given a target lens position $\mathbf{p}^*$ and rotation R^* (a look-at rotation with a level horizon, or yaw/pitch/roll for pans and tilts), damped least squares iterates

$$\Delta \mathbf{q} = J^\top (JJ^\top + \lambda I)^{-1} \mathbf{e}, \quad \mathbf{e} = [\mathbf{p}^* - \mathbf{p}(\mathbf{q}); w \log(R^* R(\mathbf{q})^\top)^\vee], \quad \lambda = 10^{-3}, w = 0.3, \quad (1)$$

with a step of 0.6, at most 80 iterations from the previous joint vector, and a tolerance of 0.1 mm and 0.06° . When the warm start fails (edge of reach), eight canonical postures are tried and the solution closest to the previous joint vector is kept; without a previous vector, the one whose base yaw faces the target with the largest margin to the joint limits. Unreachable poses make the studio hold its last pose and flag it, instead of clamping into a folded posture as an early build did. A per-joint speed cap of 1.5 rad/s (3 rad/s for the whip pan) filters the commanded joints; this is the “cinematic” rate limiter of the thesis’s deployment stack, not a dynamic model.

2.3 The lens

Full-frame sensor 35.9×24 mm. Field of view $2 \arctan(24/2f)$ vertical. Depth of field from the thin-lens circle of confusion,

$$c(d) = \frac{f}{N} \frac{|d - S|}{d} \frac{f}{S - f}, \quad \text{acceptable } c = 0.03 \text{ mm}, \quad (2)$$

rendered in the feed by a Poisson-disk gather whose radius is c converted to pixels, with each tap weighted by its own circle so that sharp foregrounds do not bleed. Zoom and focus rings are servos slewing at 60 mm/s and 4 m/s with a short first-order lag; the barrel extends 35 mm over the zoom

range. The dolly zoom holds the subject’s size in the frame while the arm moves the lens along the optical axis. The textbook law $f(t) = f_0 d(t)/d_0$ does that for a point at the subject distance, but what the audience watches is a silhouette: a convex body of radius r at distance d subtends a half-angle $\arcsin(r/d)$, so its image half-width is $f r/\sqrt{d^2 - r^2}$ and holding *that* fixed requires

$$f(t) = f_0 \frac{\sqrt{d(t)^2 - r^2}}{\sqrt{d_0^2 - r^2}}, \quad (3)$$

which collapses to the textbook law as $r \rightarrow 0$. Under the point law the silhouette instead drifts as $1/\sqrt{1 - (r/d)^2} \approx 1 + \frac{1}{2}(r/d)^2$, so the textbook form is not wrong so much as it is a far-field approximation: at a portrait distance, $r/d \approx 0.05$, it costs under 0.15% and nobody could see it. These presets are macro shots — r/d runs from 0.43 to 0.27 on the 0.225 m carrot cake between 0.26 m and 0.42 m — and there the point law leaves 6.9% of breathing in the silhouette across the move. The size that enters r is the object’s measured extent in the scene (`experiments/measure_products.js`); the spec’s `height` field is metadata for the pedestal offset and is not the object’s width — the cake is 98 mm tall and 225 mm across, and framing on the wrong one put it at 1.5 frame widths, running off both sides for the whole shot. The planner tracks the silhouette width for every shot (Figure 7).

2.4 Shots

A preset is a path (static, line, horizontal orbit, vertical arc), an aim (look-at a subject or point, look-at moving between two points, yaw/pitch/roll ranges, or track the car), a lens track (focal length constant, ramped or dolly-zoom; focus automatic, fixed, ramped or a rack between subjects; aperture) and a duration with an easing (smoothstep, linear, or an exponential snap for the whip pan). Subjects are roles (*near* hero at $x = 0.40$ m, *far* at 0.74 m, extras, the car) resolved by the active product set, so every preset works on every set. The toy car drives a 0.16 m circle around (0.58, −0.25) in 12 s — a lane of its own, because a circle centred on the hero drove the car straight through it; the tracking preset aims at its true position, and the studio offers a pixel tracker (red-blob centroid on a 132×88 read-back of the feed, proportional gaze control) as the alternative.

2.5 Framing

A preset says where the camera goes; what makes it a *shot* is how much of the frame the subject fills. For a subject of width W and height H seen from distance d , a focal length f (mm) on a 35.9×24.0 mm sensor gives an image fraction

$$\text{frac} = \frac{f}{1000} \max\left(\frac{H}{0.024}, \frac{W}{0.0359}\right) \frac{1}{d}, \quad (4)$$

the max picking whichever axis binds first. Equation (4) has three free quantities and the arm fixes one of them: on a 0.7 m arm holding a look-at orientation, the distance at which the lens can stand is not a choice, it is whatever the reachable set allows. So the distance is measured, and the *focal length* is solved: $f = 1000 \text{frac}^* d / \max(H/0.024, W/0.0359)$ with a target $\text{frac}^* = 0.55$ for a product shot and 0.24 for the tracking presets, where the subject is a 50 mm toy car that should sit inside its surroundings rather than fill the frame.

Every control point is then checked against four conditions along the whole path, at 30 Hz: the arm reaches the pose with a look-at orientation and a residual under 4 mm; no joint exceeds 95°/s; the lens stays at least 100 mm clear of every product standing on the desk, each treated as

an upright cylinder of its measured width and height; and the subject never comes closer than the lens’s 200 mm minimum focus distance. A control point that fails is pushed out along its own line of sight and re-searched over azimuth, elevation and distance, and the first pose satisfying all four is kept; a preset whose re-placement cannot be made feasible reverts to its original geometry and only its focal length is changed. Section 3.3 reports what this changed.

2.6 The planner

`plan.shots.py` samples each preset at 30 Hz, solves the IK along it, unwraps the continuous joints, differentiates for joint speeds and accelerations, and calls `mj_inverse` on the thesis model (contacts disabled; the 1.5 kg camera payload is the model’s own `ee_mount` mass) for the joint torques. It reports per shot the IK residuals, the maximum joint speed and which joint, the peak torque and which joint, the closest approach to a joint limit, the lens ranges and the subject-size statistics, and exports the joint tracks for the page. Presets that failed the first drafts were re-sited with small parameter sweeps (Section 5).

2.7 Recording and learning

The studio records episodes at 10 Hz: joints, focal length, focus distance and a 24×16 grey thumbnail of the feed, tagged with the shot and the product positions. The in-tab policy is a VisionJointPlanner-lite in TensorFlow.js: five past states and the thumbnail through a two-layer CNN, a goal token (shot one-hot plus the hero and far positions), a 256-wide MLP with layer normalisation, and the next five joint and lens deltas (action chunking); at run time ACT-style temporal ensembling averages the chunks predicted at previous steps with weights $e^{-0.35j}$.

2.8 The interface

The studio is used by someone framing a shot, not by someone commanding a manipulator, so the interface names things the way a crew does: stops rather than keyframes, *how gentle* rather than an easing exponent, *blur behind the subject* rather than an f -number (the f -number is still shown, small, beside it). A shot is a list of stops along a strip at the bottom of the screen, each carrying the pose, the aim point, the focal length, the aperture and a hold; playing interpolates position and aim with a centripetal Catmull–Rom spline and blends between a linear and a fully eased time base by a single *smoothness* number. Every stop is drawn in the set as a small wire frustum with a plane showing what the camera would see there, rendered from that pose; clicking one moves the playhead to it and, on request, flies the viewport behind the lens so the framing can be adjusted in the first person. Dragging the ball at the camera moves the selected stop and re-solves the arm live.

Focus is measured rather than assumed. The scene is re-rendered from the lens with a depth material into a 96×64 target; unpacking the RGBA-encoded depth gives z_{ndc} , and

$$z_{\text{view}} = \frac{2nf_p}{f_p + n - (2z_{\text{ndc}} - 1)(f_p - n)} \quad (5)$$

turns it into metres (n , f_p the near and far planes). A patch around the subject is sampled and the 35th percentile taken, which leans to the near side so that background showing between a subject’s edges cannot drag focus to the back wall. The lens then racks there at the servo’s rate. Autofocus drives the lens only when nothing else has a plan for it — the operator’s own shot, or a preset that asked for focus on its subject; a preset carrying a written focus track, such as the rack, keeps it, so the recorded clips remain the planned shots.

2.9 Video capture

`record.videos.js` drives the app in headless Chrome on the GPU through puppeteer, steps the simulation deterministically at 30 fps through `window.IRIS.stepFrame`, screenshots each frame of the film layout (the set full-bleed with the camera’s picture inset, 1280×720), encodes with `ffmpeg`, and stores the per-frame telemetry (joints, lens, IK status and residual, the aim point’s position in the frame) that Section 3.5 analyses.

3 Experiments

3.1 Lens model

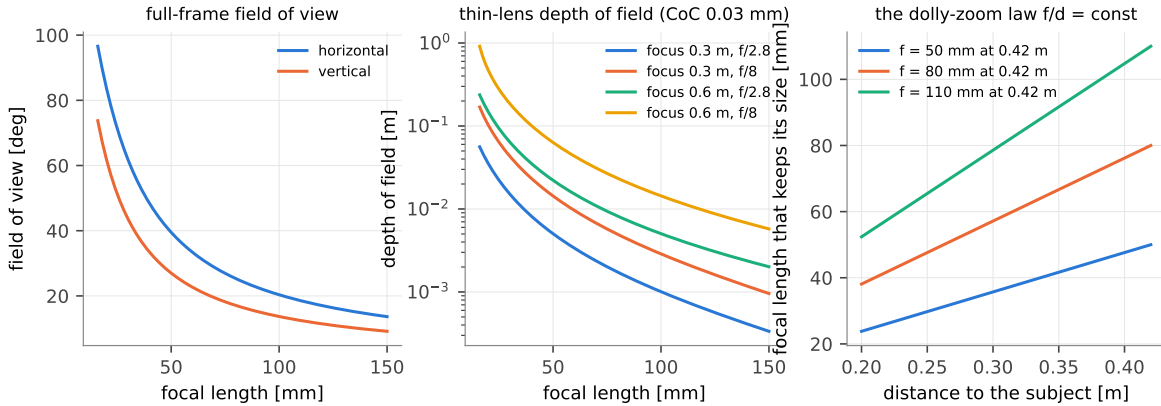


Figure 1: Field of view over the zoom range; depth of field for two focus distances and apertures; and the dolly-zoom law.

3.2 Workspace and the shot library

Table 1 is the library as shipped: 23 presets, 178s of motion, 0 unreachable samples, 3 presets above the comfort cap (the whip pan, by design), the largest speed $122^\circ/\text{s}$ and the largest torque 14.4 N m, at the shoulder as the thesis’s sizing predicts (25.8 N m worst case static with a 1.5 kg payload at full extension, which no preset reaches).

3.3 Framing, clearance and exposure

The library as first written failed its own specification in 9 of 23 presets (Figure 5). Five swept the lens *through* a product — the worst by 45 mm of overlap, which on the real rig is a collision, not a framing error. Two came closer to their subject than the lens can focus: the hero push-in ended 74 mm from the coffee cup, at which distance the cup covers 2.1 frame widths and is 126 mm inside the minimum focus distance. The frame fraction across the library ran from 0.24 to 2.13. After the re-solve, 0 presets violate any of the four conditions, the frame fraction runs 0.24–0.72 with a median of 0.55, the closest approach to a subject is 0.26 m, and the smallest clearance to a product is 0.10 m.

The lighting was rebuilt in the same pass, because measuring the pictures showed the exposure was as wrong as the framing (Figure 6). A hard 26 intensity key on a saturated warm desk, tone-mapped at 0.9 exposure, clipped 11.6% of the average frame to pure white — 47% in the worst

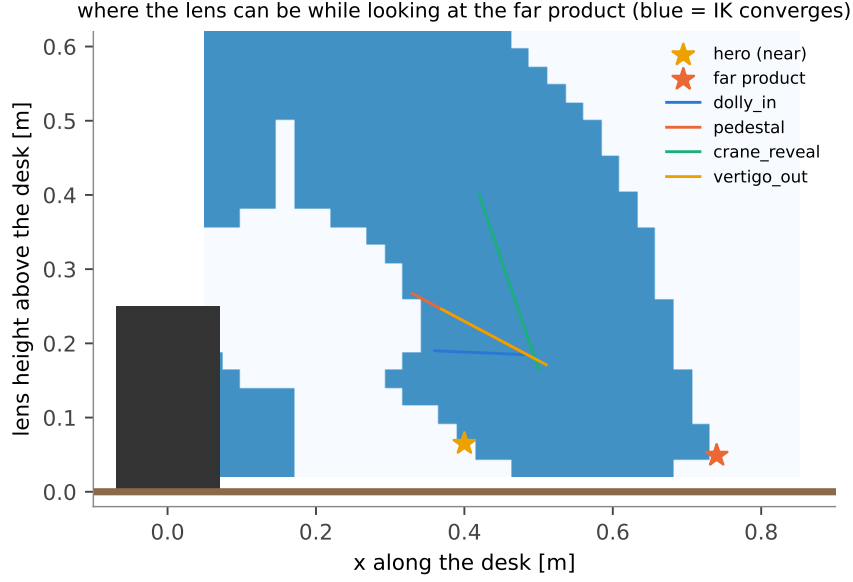


Figure 2: Lens positions in the vertical plane of the desk from which the far product can be looked at (IK converges), with four planned paths.

Table 1: Every preset through the planner (30 Hz): duration, focal length, maximum joint speed, peak inverse-dynamics torque, IK residuals, and the closest approach to a joint limit (continuous joints excluded).

shot	s	mm	$\dot{q}_{\max}$ [°/s]	$\tau_{\max}$ [N m]	pos [mm]	rot [°]	limit [°]
Pan	5	30	48	9.7	0.10	0.06	17
Tilt	5	30	21	6.9	0.10	0.06	13
Dutch roll	5	34	19	5.9	0.07	0.06	9
Zoom in	6	18–80	0	5.8	0.06	0.00	2
Zoom out	6	18–80	0	5.8	0.06	0.00	2
Dolly in	6	30	8	8.7	0.10	0.01	5
Dolly out	6	30	8	8.9	0.10	0.01	5
Truck (lateral)	6	27	109	14.4	0.10	0.02	26
Pedestal (crane)	6	33	14	9.1	0.10	0.01	1
Orbit 90°	8	35	84	13.3	0.10	0.03	24
Orbit 100°	14	38	92	13.6	0.10	0.03	14
Turntable 360°	14	28	0	7.1	0.06	0.01	7
Top view to side view	8	26	12	13.5	0.10	0.02	4
Crane reveal	7	24	29	9.5	0.10	0.03	34
Dolly zoom (out + zoom in)	8	23–39	8	9.7	0.10	0.01	8
Dolly zoom (in + zoom out)	8	21–37	8	9.6	0.10	0.01	8
Rack focus	8	44	0	5.9	0.05	0.01	11
Object tracking	12	45	6	10.0	0.05	0.06	8
Follow the car	12	40	26	6.4	0.10	0.05	6
Low reveal	6	22	13	7.0	0.10	0.01	18
Whip pan	4	21	122	13.2	0.08	0.06	13
Arc + zoom	12	26–52	81	13.7	0.10	0.03	16
Hero push-in	6	25–38	31	13.2	0.10	0.03	2

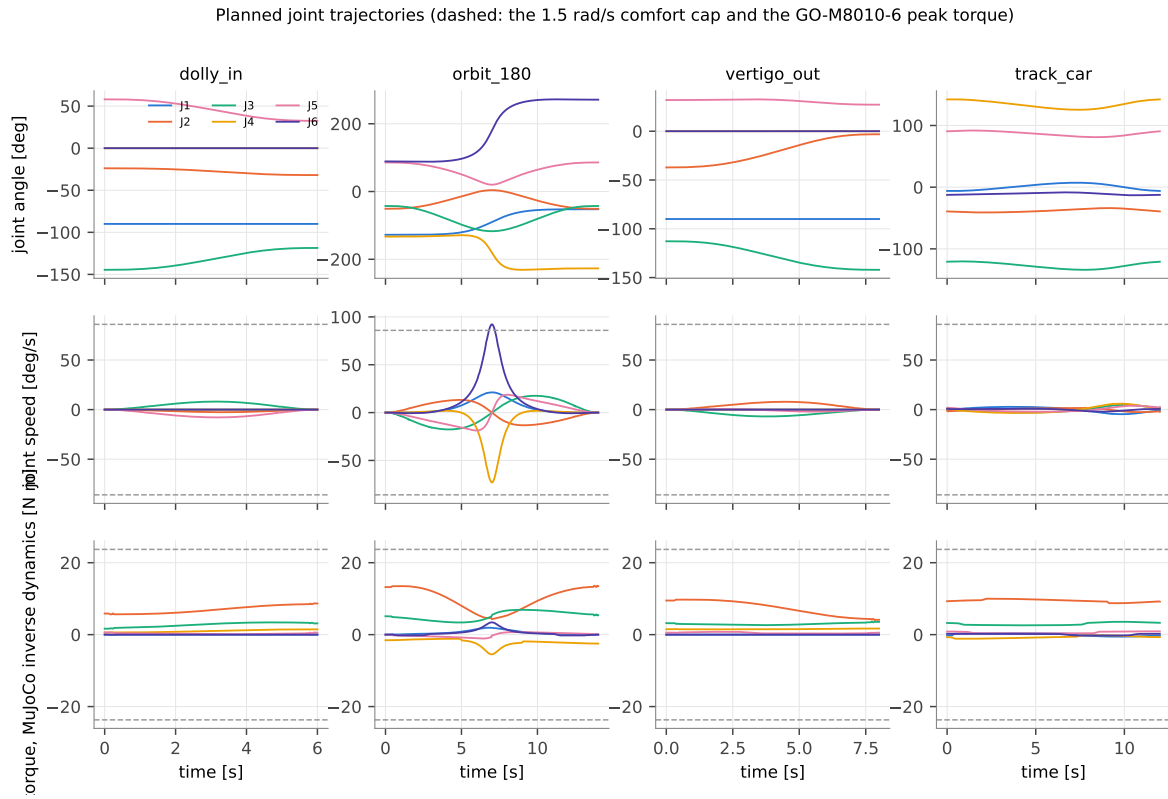


Figure 3: Four presets: joint angles, speeds against the 1.5 rad/s cap, and torques against the 23.7 N m peak.

Every preset through the planner: within the speed cap, the torque limit, and clear of the joint limits

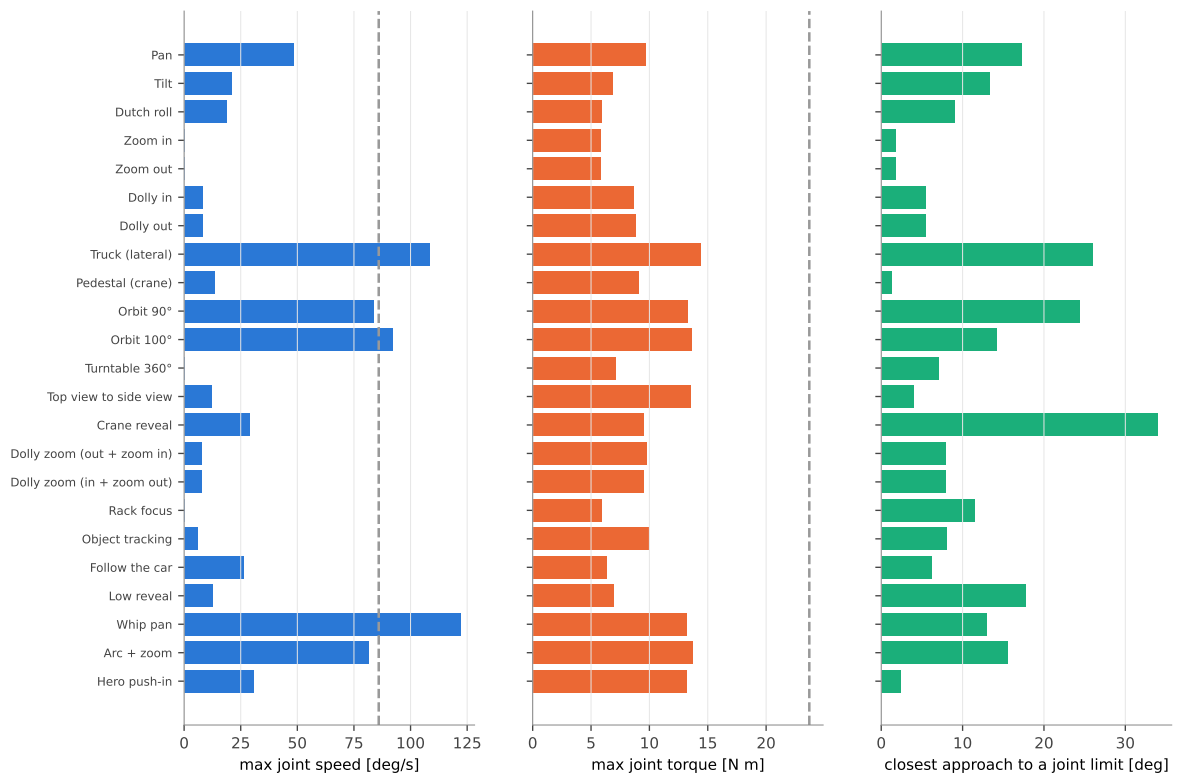


Figure 4: Per-shot maxima over the library.

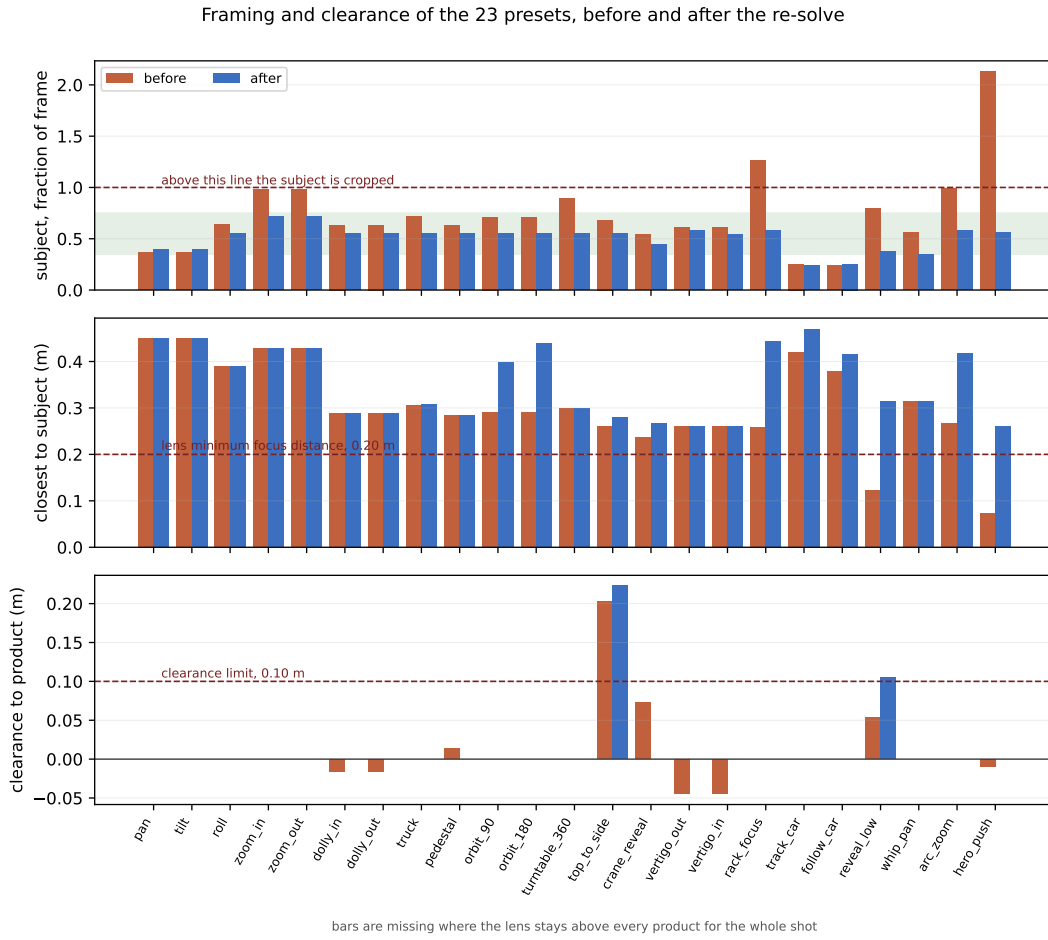


Figure 5: The 23 presets before and after the re-solve of Section 2.5: the fraction of the frame the subject fills at the middle of the move, the closest the lens comes to its subject against the lens's minimum focus distance, and the closest approach to any product on the desk. Bars are absent in the bottom panel where the lens stays above every product for the whole shot.

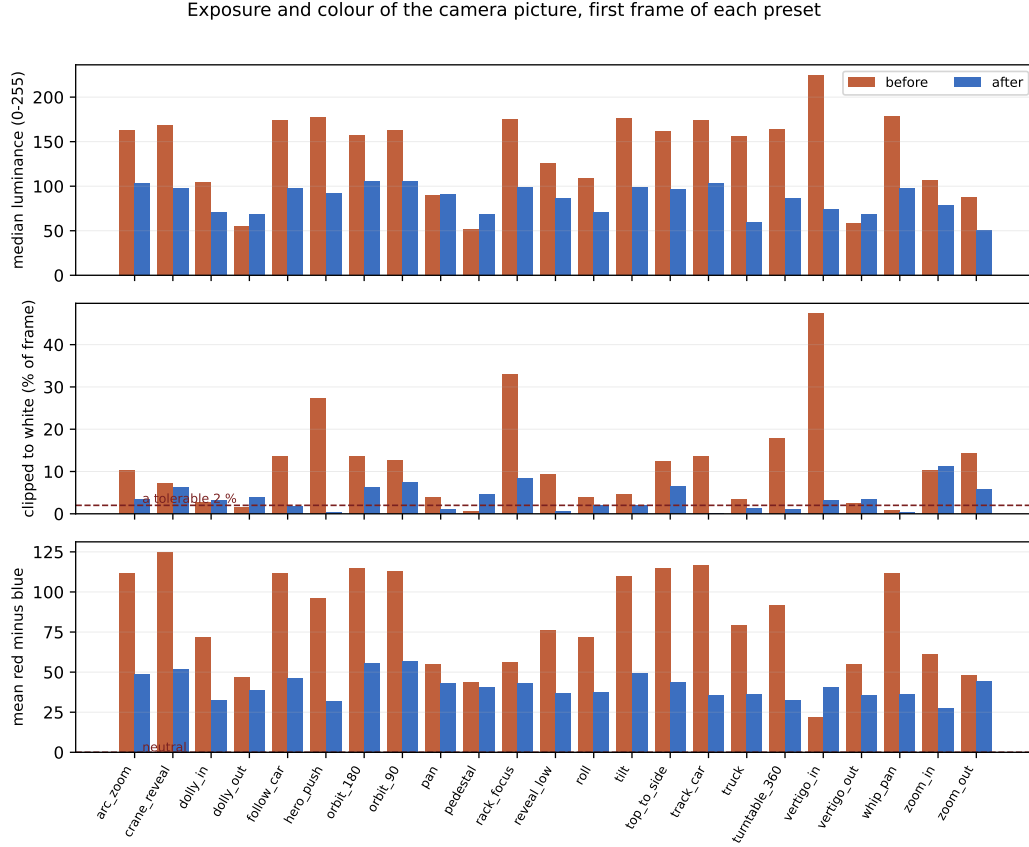


Figure 6: Exposure and colour of the camera’s own picture, first frame of every preset, before and after the lighting was rebuilt. Each frame is resampled to 180×124 ; *clipped* counts pixels at $R > 248$ and $G > 240$, and red minus blue is how a colour cast shows up numerically.

preset, which is a white rectangle with a hint of cake in it — and carried a mean red-minus-blue of 83, an orange cast strong enough to read as a fault. Softer, cooler and dimmer lights, a desaturated desk, paper-cup materials that are not pure white, a seamless cylindrical cyclorama in place of three flats that met in a visible corner, and 0.64 exposure bring this to 3.7 % clipped and a cast of 41, at a median luminance of 92 out of 255.

3.4 Dolly zoom

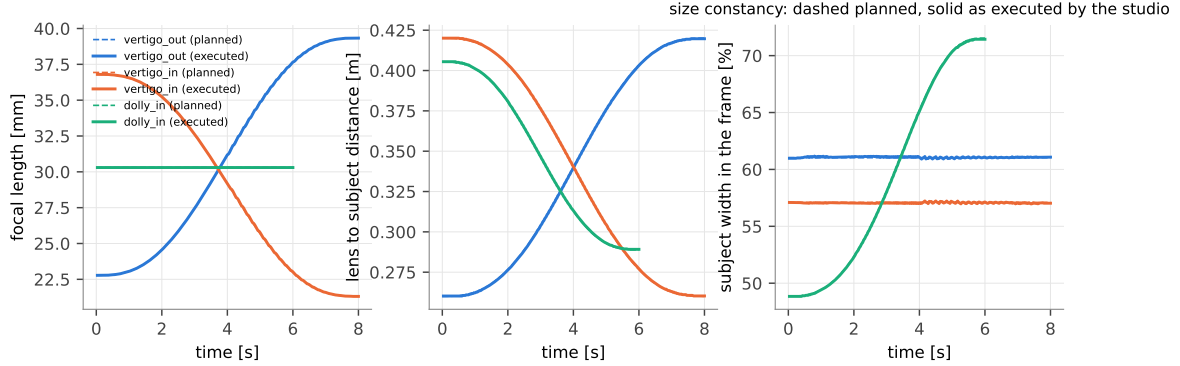


Figure 7: Focal length, lens distance and the subject’s silhouette width along the two dolly zooms and a plain dolly.

Both dolly zooms travel from 0.26 m to 0.42 m along the optical axis, 24 mm to 42 mm at $f/16$, with the cake at 65 % of the frame width. As executed — servo lag, joint tracking and the discrete focus of the lens model included — the subject’s width in the frame varies by 0.1 % (out + zoom in) and 0.1 % (in + zoom out), against 14.6 % for the plain dolly over the same travel.

That number is still the model checking itself. As an independent test, `experiments/measure_size_constancy.py` segments the cake out of the rendered camera feed of each recorded clip (warm, mid-dark connected components, the thin HUD rules rejected by area) and measures its silhouette in pixels: median 698px with a 5–95 % spread of 2.4 % for the dolly zoom in, 700px and 2.3 % out, against 888px and 38.5 % for the plain dolly (Figure 8). The residual few per cent is real and unavoidable: the cake is not a sphere, so its silhouette is not exactly the tangent circle Equation 3 assumes, and its top face foreshortens as the lens rises. The occasional single-frame outlier is the segmentation losing a fragment where the specular band crosses the cake.

3.5 Executed telemetry

Over 5739 recorded frames of 23 shots the live solver reports a mean position error of 0.26 mm (maximum 0.5 mm at the fast whip) and 0 frames out of reach; the aim point sits 0.05 % of the frame from the centre (median per shot, averaged over shots; the maximum, 19.9 %, is the whip pan’s snap), which is the difference between the commanded look-at and the rate-limited joints. 0 frames whose aim point lay outside the frustum while the arm settled into its first pose are excluded.

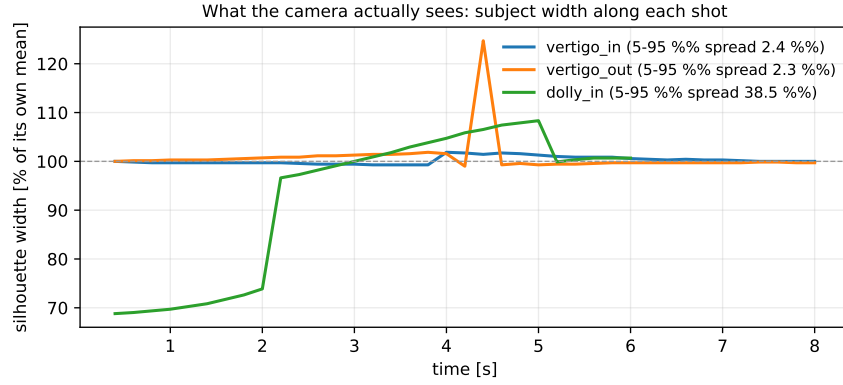


Figure 8: Subject silhouette width measured in pixels from the rendered clips, as a percentage of each clip’s own median.

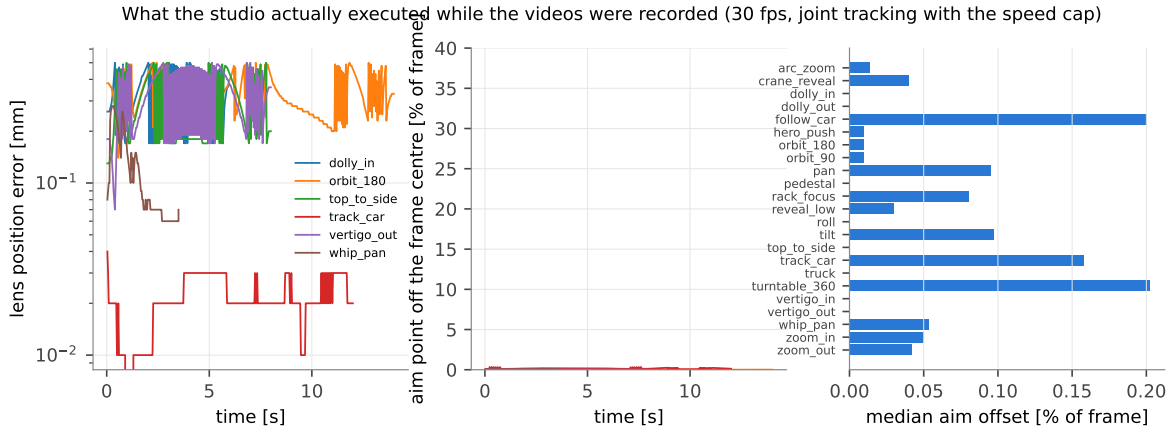


Figure 9: From the video recordings: the live solver’s lens position error and the aim point’s offset from the frame centre.

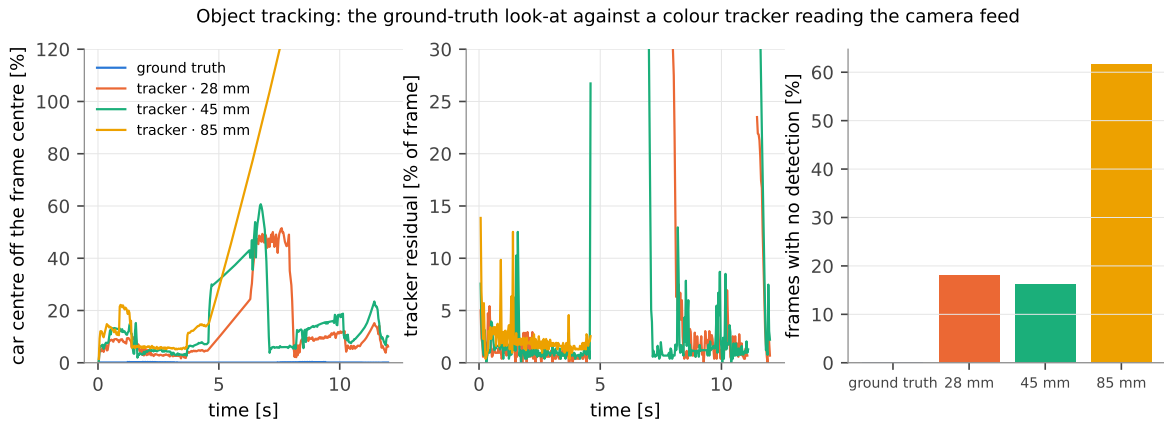


Figure 10: The toy car lapping the desk, followed two ways: from its true position (the look-at aim) and by a colour tracker reading the rendered feed, at three focal lengths.

3.6 Object tracking, ground truth against a camera-feed tracker

The tracking preset can aim either at the car’s true position or at what the camera sees. The in-browser tracker segments saturated red (the chromaticity $r/(r+g+b)$ separates the car’s paint, ≈ 0.85 , from the carrot cake’s brown crust, ≈ 0.65 , which a brightness-ratio test does not), takes connected components of a 132×88 read-back of the feed, scores them by redness rather than by size, and steers the optical axis with a proportional law applied to the camera’s *measured* pose, $\Delta\text{yaw} = -k \arctan(2u \tan(\theta_h/2))$ with $k = 1$; a blob touching the frame border is followed at $k = 0.3$ because its centre is biased outward. With ground truth the car stays 0.1 % of the frame from the centre. The tracker holds it to 13.4 % at 28 mm and 15.5 % at 45 mm, with residuals (what it still sees off-centre) of 9.5 % and 6.0 %. At 85 mm the field of view is only 16° and the loop fails: the car leaves the frame, the tracker has no search behaviour, and it is lost for 62 % of the frames. Two attempts to lead the target with an estimated angular rate made this worse, because the rate had to be estimated from the controller’s own corrections and the loop then amplified them; the honest conclusion is that a reactive colour tracker needs either a wider lens or a real state estimator.

3.7 Learning a shot from demonstrations

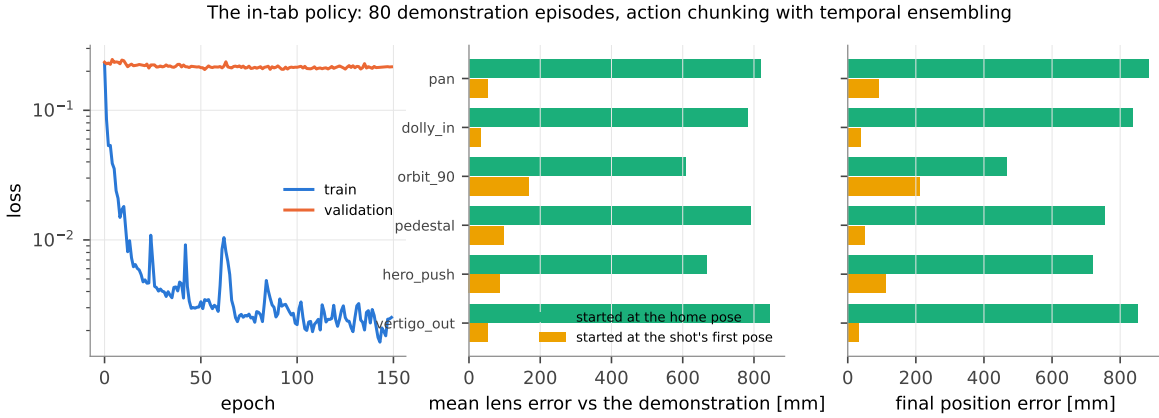


Figure 11: The in-tab policy: training curve, and rollouts from the home pose against the demonstration of the same shot.

The studio records demonstrations of its own presets and trains the VisionJointPlanner-lite of Section 4 in the browser. With 80 episodes (every non-tracking preset on all four product sets) and 150 epochs the training loss reaches 0.0025 while the held-out loss stays at 0.22: TensorFlow.js validates on the tail of the data, so the held-out set is whole shots the model never saw, and the policy does not generalise across shots at this scale. Rolled out it reproduces the beginning of a move and then drifts: started at the home pose, which no demonstration ever visits, it ends 752 mm from where the demonstration ended; started at the shot’s own first pose, as a deployment would after the thesis’s goal-directed start, 89 mm. This is the behaviour the thesis met before scaling its dataset (17,897 samples against the few thousand here) and before adding the goal-directed start and stagnation detector of its deployment stack; it is included as a working data-collection and training loop, not as a result.

4 Implementation and usage

experiments/irisstudio/kinematics.py (Arm, tool frame, IK, seeds), shots.py (the evaluator), plan_shots.py, tune_vertigo.py (re-sites the dolly zooms against IK, lens range and depth of field), measure_products.js (the products' real extents), record_videos.js, measure_size_constancy.py (the pixel check of Figure 8), measure_framing.py (Equation (4), distances and clearances for a spec), contact_sheet.js and measure_exposure.py (the picture statistics of Figure 6), make_figures.py; the spec out/iris_studio.json is copied to the site as assets/data/iris_studio.json. Browser: assets/js/iris/kin.js, lens.js, shots.js, scene.js (three.js: desk, arm from the decimated STL links, camera rig, products, lights), dof.js, policy.js, timeline.js (stops and interpolation), studio_app.js (the interface of Section 2.8); app.js is the previous single-page controller, kept for reference. Pages iris_studio.html (styled by assets/css/iris-studio.css) and iris.html.

```
cd experiments && python -m pytest -q tests
node measure_products.js          # out/product_sizes.json (real extents)
python tune_vertigo.py --apply    # re-sites the two dolly zooms
python plan_shots.py             # out/shots_plan.json (-> assets/data/)
node record_videos.js            # out/video/iris_<shot>.mp4 + telemetry
python measure_size_constancy.py  # pixel check on the clips
python measure_framing.py --spec <old> --out ../out/framing_before.json
python measure_framing.py --out ../out/framing_after.json --figure
node contact_sheet.js --out ../out/sheet_after
python measure_exposure.py --dir ../out/sheet_after --figure
python make_figures.py           # figures, numbers.tex, shots_table.tex
cd ../docs && make
```

The two node steps need `python3 -m http.server 8765` running in the site root. Hardware path: the planner's joint tracks are 30 Hz position setpoints in the model's joint order; the thesis's ROS node consumes exactly that over RS-485, and the lens tracks are two servo setpoints per frame.

5 Lessons and limitations

- Orbits at desk height fail: looking back toward the base needs the wrist to fold past its $\pm 100^\circ$ pitch. The turntable orbit lives high above the product and looks down; half orbits stay on the base side.
- **The widest horizontal orbit this arm can carry is 100° .** Sweeping the search over span, radius, height and centre subject, and requiring reach, a $95^\circ/\text{s}$ joint-speed ceiling and 100 mm of clearance from every product, no solution exists at 120° or beyond: the wrist runs out of twist while holding the horizon level. The library therefore has a 90° and a 100° orbit and no 180° one, and a full turn is done by rotating the product instead.
- **A framing target is a constraint on the arm, not on the lens.** Placing cameras by eye and picking focal lengths afterwards put 9 of 23 presets outside a limit, including five whose lens swept through a product and a push-in that ended 74 mm from a coffee cup. Solving Equation (4) for f at whatever distance the arm can hold, with reach, speed, clearance and minimum focus all checked along the path, removes every violation without hand-tuning.

- **Measure the picture, not just the trajectory.** The trajectories were clean while a quarter of the average frame was clipped to white and the whole library carried a strong orange cast; nothing in the joint-space telemetry says so. Two summary statistics per frame — the clipped fraction and red minus blue — caught it immediately and turned the lighting from a matter of taste into a number to drive down (Figure 6).
- A straight-down top view sits on the look-at singularity (the horizon has no meaning); the top-to-side arc starts at 70° elevation and comes in from the side.
- Any lens position closer than about 0.3 m to the base column folds the arm onto itself once the tool offset is included; the first drafts of the dollies started there.
- The wrist singularity ($q_5 \rightarrow 0$) makes joints 4 and 6 swing quickly when the optical axis passes through the forearm axis; the tracking presets were re-sited to keep $|q_5|$ away from zero, and continuous joints are unwrapped.
- Product clearance is checked against upright cylinders, not meshes, and only for the lens’s entrance pupil — the links behind it are not swept. Self-collision is still not modelled, so a pose that clears the products may not clear the arm’s own forearm.
- Not modelled: motor dynamics and belt compliance, self-collision, lens breathing and the real minimum-focus curve, a physically calibrated exposure (the numbers here are relative, and only comparable between builds of this renderer). The in-tab policy is a toy; the thesis’s numbers stand.

References

- [1] Q. Cheng, “IRIS: Intelligent Robotic Imaging System,” MEng thesis, University of Toronto, 2025.
- [2] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed., Pearson, 2005.
- [3] S. R. Buss, “Introduction to inverse kinematics with Jacobian transpose, pseudoinverse and damped least squares methods,” technical report, UCSD, 2004.
- [4] T. Z. Zhao, V. Kumar, S. Levine and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” *RSS*, 2023.
- [5] E. Todorov, T. Erez and Y. Tassa, “MuJoCo: A physics engine for model-based control,” *IROS*, 2012.
- [6] M. Potmesil and I. Chakravarty, “A lens and aperture camera model for synthetic image generation,” *SIGGRAPH*, 1981.